

## *Full Project Overview*

### *AI Coding Question Recovery & Practice Platform*

## **1. Introduction**

This project is an AI-powered coding practice and recovery platform designed for students, interview candidates, and competitive programmers who want to revisit and solve coding questions from online assessments, coding contests, interview rounds, and campus exams.

The platform solves a common problem faced by students during coding assessments:

- They fail to solve a question during the test,
- later they only have a screenshot of the problem,
- And there is no easy way to reconstruct the full coding environment for practice.

This platform provides a complete workflow where a user can upload a screenshot of a coding question and instantly receive:

- a reconstructed problem statement,
- structured examples,
- generated test cases,
- an integrated coding compiler,
- execution verdicts,
- and AI-powered feedback.

The system combines:

- OCR,
- AI reasoning,
- compiler execution,
- test validation,

- and intelligent developer tooling into one unified platform.
- 

## 2. Problem Statement

Students regularly participate in:

- campus coding exams,
- online assessments,
- DSA contests,
- interview rounds,
- and practice platforms.

Many times they:

- cannot solve the question within the exam time,
- forget the problem later,
- only keep screenshots,
- do not know hidden edge cases,
- and cannot recreate the same testing environment.

Existing coding platforms like:

- [LeetCode](#)
- [HackerRank](#)
- [Codeforces](#)

allow users to solve problems,  
but they do not allow:

- screenshot-based problem reconstruction,
- AI-generated test cases,
- intelligent recovery practice,
- or AI-powered debugging workflows.

This project fills that gap.

---

## 3. Project Vision

The vision of this platform is:

“To create an AI-assisted coding recovery system that helps users reconstruct, practice, analyze, and improve failed coding questions from assessments.”

The platform is not intended to become:

- another competitive coding platform,
- or a simple online compiler.

Instead, it focuses on:

- intelligent problem reconstruction,
  - realistic practice environments,
  - and AI-assisted learning workflows.
- 

## 4. Core Features

The platform contains multiple integrated systems working together.

---

### Feature 1 — Screenshot Upload System

Users can upload:

- coding question screenshots,
- OA screenshots,
- contest screenshots,
- interview question images.

The system validates:

- file type,
- image quality,

- supported formats.
- 

## **Feature 2 — OCR Extraction Engine**

The uploaded screenshot is processed using OCR.

The OCR engine extracts:

- problem text,
- constraints,
- examples,
- input/output formats.

The raw OCR output is cleaned and normalized for AI processing.

---

## **Feature 3 — AI Problem Reconstruction**

The extracted text is passed to an AI reasoning engine.

The AI converts messy OCR text into a structured coding problem.

The reconstructed problem contains:

- title,
- clean description,
- examples,
- constraints,
- input format,
- output format.

This creates a proper coding question environment automatically.

---

## **Feature 4 — Integrated Compiler System**

The platform provides a built-in coding compiler where users can:

- write code,
- execute solutions,
- test outputs,
- debug logic.

Supported languages:

- C++
- Java
- Python
- JavaScript

The compiler system behaves similarly to modern coding platforms.

---

## Feature 5 — Test Case Engine

The platform generates and validates:

- sample test cases,
- hidden test cases,
- edge cases,
- stress cases.

This system ensures that:

- solutions are tested properly,
- hardcoded answers fail,
- inefficient solutions are detected.

This is one of the most important parts of the platform.

---

## Feature 6 — Validation & Verdict Engine

The validation system checks:

- correctness,
- runtime,
- memory usage,
- execution failures.

Supported verdicts:

- Accepted
- Wrong Answer
- Runtime Error
- Compilation Error
- Time Limit Exceeded
- Memory Limit Exceeded

This creates a realistic coding judge experience.

---

## **Feature 7 — AI Feedback Engine**

If the user's solution fails, the AI system analyzes:

- failed test cases,
- logic issues,
- possible edge cases missed,
- optimization opportunities.

Instead of only returning:

“Wrong Answer”

The platform provides intelligent debugging guidance.

This becomes a major standout feature

---

## **Feature 8 — Analytics Dashboard**

The platform tracks:

- solved questions,
- weak topics,
- runtime performance,
- submission history,
- topic-wise accuracy.

This allows users to improve continuously.

---

## 5. Complete User Workflow

The entire platform works as follows:

User Uploads Screenshot



OCR Extracts Text



AI Reconstructs Problem



Structured Problem Created



User Writes Code



Compiler Executes Code



Hidden + Visible Test Cases Run



Validation Engine Checks Output



Verdict Generated



AI Feedback Returned



Submission Stored in Dashboard

---

## 6. System Architecture Overview

The platform consists of multiple systems connected together.

---

### Frontend Layer

Responsibilities:

- dashboard UI,
  - code editor,
  - upload system,
  - result visualization,
  - analytics dashboard.
- 

### Backend Layer

Responsibilities:

- API handling,
  - authentication,
  - OCR coordination,
  - compiler orchestration,
  - business logic,
  - validation systems.
- 

### AI Layer

Responsibilities:

- OCR text understanding,
- problem reconstruction,
- test case generation,
- hint generation,



- edge case analysis.
- 

## Compiler Layer

Responsibilities:

- compile user code,
  - execute submissions,
  - sandbox execution,
  - capture runtime results.
- 

## Database Layer

Responsibilities:

- store users,
  - problems,
  - submissions,
  - test cases,
  - analytics.
- 

## 7. Technology Stack

### Frontend

- Next.js
  - TypeScript
  - Tailwind CSS
  - Monaco Editor
-

## Backend

- FastAPI
  - Python
  - SQLAlchemy
- 

## Database

- PostgreSQL
- 

## OCR

- EasyOCR
- 

## AI

- Gemini API / OpenAI API
- 

## Compiler Engine

- Judge0 API
- 

## Storage

- Cloudinary
- 

## Deployment

- Vercel
  - Railway
  - Neon PostgreSQL
- 

## 8. Project Architecture Philosophy

The project will follow:

### Modular Architecture

Each system will be separated into:

- independent modules,
- isolated services,
- reusable components.

This improves:

- scalability,
  - maintainability,
  - debugging,
  - future feature expansion.
- 

## 9. Backend Development Philosophy

The backend will follow:

Routes Layer



Service Layer



Business Logic



Database Layer

This prevents:

- tightly coupled logic,
  - messy APIs,
  - unmaintainable codebases.
- 

## **10. Development Strategy**

The project will be developed incrementally.

Instead of building everything together, development will happen phase-by-phase.

---

### **Phase 0 — Backend Foundation**

Setup:

- project structure,
  - FastAPI,
  - PostgreSQL,
  - environment management,
  - exception handling.
- 

### **Phase 1 — OCR Pipeline**

Build:

- screenshot upload,
  - OCR extraction,
  - OCR cleanup.
- 

### **Phase 2 — AI Problem Parsing**

Build:

- AI structured parsing,
  - JSON reconstruction,
  - constraints extraction.
- 

## **Phase 3 — Compiler Engine**

Build:

- Judge integration,
  - code execution,
  - output capture.
- 

## **Phase 4 — Test Case System**

Build:

- visible test cases,
  - hidden cases,
  - validation logic.
- 

## **Phase 5 — AI Features**

Build:

- edge case generation,
  - AI hints,
  - optimization suggestions.
- 

## **Phase 6 — Dashboard & Analytics**

Build:

- submissions,
  - topic tracking,
  - progress analytics.
- 

## 11. MVP Scope

The initial MVP will focus only on:

- screenshot upload,
- OCR extraction,
- structured problem creation,
- basic compiler execution,
- visible test case validation.

The goal of the MVP is:

- validate architecture,
  - complete core workflow,
  - create stable backend foundation.
- 

## 12. Advanced Future Scope

Future improvements may include:

- Docker-based sandbox execution,
  - contest simulation,
  - AI-generated hidden test cases,
  - topic-wise AI coaching,
  - solution comparison engine,
  - collaborative coding rooms,
  - personalized DSA roadmap generation.
-

# 13. Engineering Goals

This project is designed to demonstrate:

## Backend Engineering

- scalable APIs,
  - modular architecture,
  - service separation.
- 

## AI Engineering

- OCR integration,
  - prompt engineering,
  - AI reasoning systems.
- 

## Compiler Systems

- secure execution,
  - validation engines,
  - runtime analysis.
- 

## Database Design

- relational schemas,
  - submission tracking,
  - analytics systems.
- 

## Product Thinking

- real-world problem solving,
  - developer tooling,
  - intelligent workflows.
-

## 14. Final Goal

The final product should feel like:

“An AI-powered coding recovery and practice system that intelligently reconstructs coding questions from screenshots and provides a realistic coding judge environment with AI-assisted debugging.”

The platform should be:

- scalable,
- modular,
- production-ready,
- developer-focused,
- and professionally engineered.

 [Listen to this tab](#)